Habit Meets Schedule: Re-Ranking Five TV Programmes per Viewer on sweet.tv

Mark Starchenko (individual)
HSE University, Graduate School of Business, mastarchenko@edu.hse.ru, 2026-10-24
Final exam quiz: TV programme recommendation on sweet.tv. Code: https://github.com/Marrrik/sweettv-recsys

***Abstract*—For 1,260 sweet.tv viewers we predict the five programmes they will most often watch through at least 80% of an airing in weeks 31 to 42 of 2020, from twenty weeks of channel-level viewing and a broadcast schedule in which only 10.8% of rows carry a programme id. We join views to airings by channel and time overlap, rebuild the organisers' target on a weeks 21 to 30 hold-out, and compare popularity, a habit-times-schedule heuristic, implicit ALS, personalised PageRank, association rules and content-based filtering with a LightGBM re-ranker built on all of them. The re-ranker reaches MAP@5 0.3595 on the hold-out against 0.3366 for the heuristic and 0.3184 for the best single method, and a public score of 0.45244. Most of the signal is a viewer's own habit placed on the coming schedule, and no method finds the unseen films that light viewers watch next.**

***Index Terms*—implicit feedback, TV recommendation, learning to rank, personalized PageRank, matrix factorization**

I. Introduction

The Kaggle community competition *sweet.tv: TV program recommender* [8] gives a log of which channel each user watched and when, and the schedule of what aired. The data page defines the target: per user and airing compute the watched share, keep airings watched for at least 0.8 of their length, count them per programme, and take strictly the five most frequent programmes of weeks 31 to 42, ranked by MAP@5. The quiz requires at least two of four method families, a popularity reference, one documented hold-out and a strategy a service could adopt.

A TV service needs such a list for a "your programmes" row: what to remind a viewer of, and what new to show. The task is unusual because the items are scheduled: a programme can only be watched when it airs, and the schedule of the test weeks is public.

Example. User A (test user 3525695269081370558) fully watched *SpongeBob SquarePants* on 32 airings, a second cartoon on 16 and the *TSN* news on 10 in weeks 11 to 20. The hybrid, trained on earlier snapshots, returns ids 12001682, 12002856, 2400480, 2400508, 2400467 for weeks 21 to 30; four of them are in A's actual top five (AP@5 0.71).

Contributions. (1) A leak-free reconstruction of the target on time snapshots, with a heuristic that multiplies a user's per-airing watch rate by the airings in the coming schedule. (2) A comparison of six single methods and a re-ranker on one hold-out with paired bootstrap intervals, and an account of which list properties explain the differences.

II. Related Work

Hu, Koren and Volinsky [1] built implicit-feedback ALS for exactly this setting, a TV service where r_{ui} counts how many times a user fully watched a show, with confidence $c_{ui} = 1 + \alpha \log(1 + r_{ui}/\epsilon)$ as one variant; they note that predicting a re-watch is much easier than a first watch. Personalised PageRank restarts a random walk at a user-specific preference vector [2]. Association rules score $X \Rightarrow Y$ by support and confidence [3]. Two-stage systems first generate a few hundred candidates and then rank them with a richer model [5]. LambdaMART is the boosted-tree version of LambdaRank [6]; we use the gradient boosting library LightGBM [4] and its `lambdarank` objective. Reciprocal rank fusion sums $1/(k+r)$ over rankings, with $k = 60$ fixed by its authors [7].

Alternatives considered. Sequence models over the view stream: the target counts repeated full viewings, which a per-airing rate already captures, at a fraction of the cost. Recovering programme ids for unlabelled airtime by title: only 4.4% of unlabelled airtime has a title equal to a labelled one (EDA), so it would add little. A neural two-tower retriever: the candidate stage already covers 78.6% of target pairs with simple sources.

III. Method

A. Join, watched rule and unlabelled airtime

A view matches every schedule slot on the same channel (`vsetv_id` equals `channel_id`) whose airing interval overlaps the view; the covered seconds of one airing are summed over the user's views and divided by the slot length, capped at 1. The 0.8 threshold is the organisers' rule, so our labels and our test target are defined the same way. The observed distribution supports a share-based cut over an absolute one: covered shares are bimodal (Fig. 1b), and 71% of events on slots of at most 10 minutes reach 0.8 against 7% on slots over two hours, so a fixed number of seconds would mean very different things for news and films. Rows with `tv_show_id` = 0 are not items: they never enter the pool or the targets, but all airtime, labelled or not, feeds the viewing-habit features below. The two clocks agree: view stops cluster at slot boundaries only at zero shift, in every week.

B. Snapshots, hold-out and leakage

A snapshot S_T uses views before Monday of ISO week T, clipped at that moment, as history, and the target of weeks T to the window end. The hold-out is S_{21} (history weeks 11

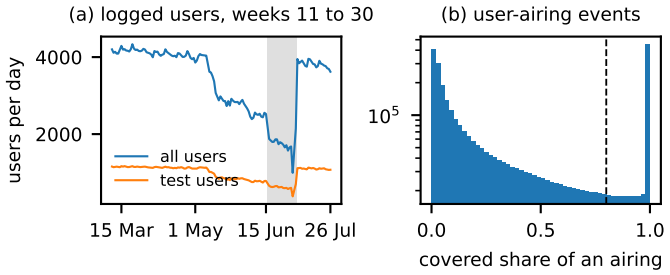


Figure 1. (a) Users with a view per day; shaded: 16 June to 4 July, 23.9% of users active before and after have no view inside it, and 3 July is empty. (b) Covered share of 3,014,129 user-airing events, log scale; dashed: the 0.8 rule (`report/make_figures.py`).

to 20, target 21 to 30, 4,030 users, 1,150 of them test users): it is the latest window the quiz allows, adjacent to the test window and close to it in length (10 weeks against 12), and its history has the same start as the final model's. Every model is refitted per snapshot on its history only; the target-window schedule tells which programme airs where and when, never who watched it. The ranker learns from S_{14}, S_{16} and S_{18}, whose labels end in week 20, with early stopping on 20% of their users, so S_{21} is used only to choose. The final models are refitted on S_{14}, S_{16}, S_{18}, S_{21} and S_{26} and predict S_{31} (history 11 to 30, target 31 to 42).

C. Common rules for every method

The pool is the labelled programmes that air in the target window; recommending an already watched programme is allowed for every method. A method's positive scores are sorted (ties by id) and the empty places are filled from one popularity ranking, the same for all methods and the whole answer for users without history: pool programmes ordered by how many users have them in their history top five, then by airings in the window. Among test users 4 have no fully watched airing in weeks 11 to 30 and get this list, and 15 have one to four such programmes and get it after their own. Every list therefore has exactly five distinct ids. Scores of different methods are never compared directly.

D. Single methods

Popularity is that fallback ranking alone. *Habit × schedule* scores

$$s(u,p) = \sum_{c} \min\Big(1, \frac{q_w(u,p,c)}{e_w(u,p,c) + \pi}\Big)\, A(p,c),$$

where q_w counts the airings of p on channel c that u watched for 0.8, e_w the airings of p on c on days u was in the log (both weighted with a 10-week half-life), $\pi = 30$ and A the airings of p on c in the window. Counting only logged days keeps rates comparable across the logging gap of Fig. 1a. *ALS* follows [1] with $c = 1 + \alpha \log(1+w)$, w the recency-weighted count, 32 factors, $\alpha = 1$, $\lambda = 30$, 15 sweeps of the closed-form solves. *PPR* walks a programme graph whose edge weight is the number of users who fully watched both programmes, row-normalised, follows an edge with probability 0.5 and restarts at the user's normalised $\log(1+w)$ history. *Association rules* score C by $\max_A P(C \mid A)$ over the user's programmes A, for pairs shared by at least 30 users. *Content-based* builds TF-IDF vectors from category, genres, decade, directors and actors and ranks by cosine similarity to the user's weighted profile.

E. Hybrid: candidates and re-ranking

Candidates are every pool programme the user touched in the history, the top 60 of popularity, ALS, PPR, rules and of a habit-slot score, and the top 20 of content. The habit-slot score multiplies a user's share of each channel and hour of the week spent watching (all airtime) by the airings a programme has in those cells. The ranker sees 67 features: the habit × schedule score and two variants, counts, rates and recency of the user's views of the programme, window and history airings, habit-slot exposure with each airing spread over all the hours it covers, programme popularity, category and year, user activity, every method's score and its rank within the user. Labels mark membership in the target five. LightGBM 4.7.0 is trained with the binary and the `lambdarank` objectives (one group per user and snapshot) and three seeds; the six models are fused by summing $1/\mathrm{rank}$ within the user, which is reciprocal rank fusion [7] with $k = 0$ instead of its authors' 60. Weights are equal and not tuned.

F. Configuration

LightGBM: learning rate 0.03, 31 leaves, 300 rows per leaf, feature fraction 0.6, bagging 0.8, L2 10, final rounds 1.2 times the early-stopping optimum. Seeds: 0 for ALS, sampling and bootstrap, 0 to 2 for LightGBM. Environment: Windows 11, AMD Zen 4 laptop CPU (16 threads), 15.2 GB RAM, Python 3.12.14, NumPy 2.5.4, pandas 3.0.6, SciPy 1.18.1, scikit-learn 1.9.1. Runtimes: EDA 72 s, single methods 215 s, final re-ranker notebook 1,488 s including all six snapshots.

IV. Experiments

A. Setup

The log holds 5,821,987 views of 4,914 users on 202 channels from 9 March to 26 July 2020; 10.8% to 11.3% of schedule rows and 16.0% to 16.9% of airtime carry an id. 94.1% of viewing time falls inside some slot and 53.1% inside labelled slots. Of 3,014,129 user-airing events, 615,979 (20.4%) reach 0.8. MAP@5 is averaged over the hold-out target users; differences use a paired bootstrap over users (2,000 resamples). Kaggle scores are the returned late-submission scores.

B. Results

Table I compares the methods. Habit × schedule beats every single method and beats the count of fully watched airings alone (0.3095) by 0.027; the hybrid beats it by 0.0229 [0.0197, 0.0261], higher for 1,452 users and lower for 849. The order of the methods is the same on S_{21}, on S_{16} and on the public score. Kaggle scores are higher than hold-out scores for every method; the test window is longer (12 weeks against 10) and

Table I

SINGLE METHODS AND THE FINAL HYBRID. MAP@5 ON THE HOLD-OUT S_{21} (ALL 4,030 TARGET USERS, AND THE 1,150 TEST USERS AMONG THEM), ON S_{16} (HISTORY WEEKS 11 TO 15, TARGET 16 TO 20), AND THE RETURNED KAGGLE SCORES. LAST COLUMN: HYBRID MINUS METHOD ON S_{21} WITH A 95% PAIRED BOOTSTRAP INTERVAL (`MODELLING/ANALYSIS.IPYNB`).

Method	S_{21} all	S_{21} test users	S_{16} all	public	private	hybrid − method on S_{21}
Popularity	0.2341	0.2592	0.2478	0.32330	0.31620	+0.1254 [0.1192, 0.1319]
Habit × schedule	0.3366	0.3770	0.3484	0.42188	0.41561	+0.0229 [0.0197, 0.0261]
ALS	0.2832	0.3064	0.3128	0.36794	0.36427	+0.0763 [0.0717, 0.0812]
PPR	0.3184	0.3572	0.3382	0.40855	0.39635	+0.0411 [0.0378, 0.0445]
Association rules	0.2545	0.2793	0.2776	0.33741	0.34068	+0.1050 [0.0997, 0.1108]
Content-based	0.0854	0.0899	0.1082	0.09622	0.08950	+0.2741 [0.2669, 0.2818]
Hybrid (final)	**0.3595**	**0.3960**	(train)	**0.45244**	0.44757	–

Table II

RE-RANKER ROUNDS: TRAINING SNAPSHOTS, CANDIDATE TOP-n (SOURCES/POPULARITY), FEATURES, OBJECTIVES (B+L: BINARY AND LAMBDARANK), MAP@5 ON S_{21} AND PUBLIC SCORE OF THE SUBMITTED ONES (`MODELLING/RANKER*.IPYNB`).

Round	Train	Cand.	Feat.	Objective	all	test	public
R1	S16	30/50	63	binary	0.3539	0.3915	0.44585
R1	S16	30/50	63	lambdarank	0.3566	0.3956	0.44959
R1	S16	30/50	63	poisson	0.3510	0.3877	0.43942
R1	S16	30/50	63	RRF, all 3	0.3568	0.3947	0.44846
R2 E1	S16	60/60	63	RRF b+l	0.3554	0.3945	–
R2 E2	S14–S18	60/60	63	RRF b+l	0.3554	0.3923	–
R2 E3	S14–S18	60/60	63	RRF b+l, reg	0.3574	0.3935	0.45041
R3 E4	S14–S18	60/60	67	RRF b+l, reg	0.3586	0.3948	–
R3 E4	S14–S18	60/60	67	3 seeds, reg	0.3595	0.3960	0.45244

Table III

PROPERTIES OF THE TOP-5 LISTS ON S_{21}: SHARE OF RECOMMENDED PROGRAMMES THE USER WATCHED FOR 0.8 BEFORE, ONLY TOUCHED, OR NEVER TOUCHED; PRECISION ON FAMILIAR AND ON NEW PROGRAMMES; DISTINCT PROGRAMMES OVER ALL USERS; MEAN OVERLAP WITH THE POPULARITY FIVE; SHARE OF NEWS. IN THE ACTUAL TARGETS 50.9% / 14.5% / 34.6% ARE WATCHED / TOUCHED / NEW.

Method	watch.	touch.	new	P fam.	P new	dist.	pop.	news
Popularity	60%	22%	19%	0.358	0.068	5	5.00	80%
Habit × sched.	96%	2%	3%	0.397	0.157	482	2.43	46%
ALS	87%	6%	7%	0.367	0.093	78	2.90	54%
PPR	96%	1%	2%	0.379	0.157	730	2.35	44%
Assoc. rules	70%	18%	12%	0.352	0.065	35	3.85	70%
Content	35%	13%	51%	0.314	0.010	576	1.20	53%
Hybrid	83%	12%	5%	0.430	0.139	176	2.76	50%

has no logging gap, which we take as the likely reason without having tested it.

Table II traces the re-ranker. Larger candidate lists raised recall of the target from 75.7% to 78.6% but not MAP@5; more training snapshots and stronger regularisation gave +0.002, coverage features +0.001 and three seeds +0.001. Each step is within the noise of a single run, but together they moved both S_{21} and the public score. The largest gain shares in the final model are the within-user rank of habit-slot exposure (42%), the within-user rank of the habit-slot score (22%) and habit coverage of whole airings in the window (9%).

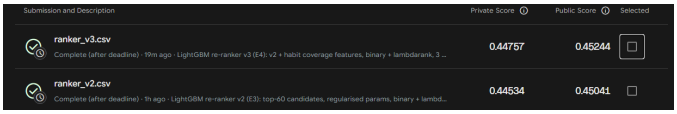


Figure 2. Returned Kaggle score of the designated final submission `ranker_v3.csv`.

C. Recommendation examples

Table IV shows the same two users for all methods. A watches cartoons and news; habit × schedule finds four of A's five (AP 0.76), the hybrid four in a different order (0.71) and PPR four (0.64). B's actual five are four films B never touched and the series *CSI*, watched once before; every method scores 0, and only content-based and rules propose anything new, all of it wrong.

D. Verification

`tests/check_core.py` checks AP@5 against hand-computed cases (hits at ranks 1 and 2 of five give 0.4, at ranks 4 and 5 give 0.13), the join on a toy schedule, the target rule, one ALS half-step against eq. (4) of [1] written out directly, and that every submission has the template's 1,260 users in the template's order with five distinct ids. The second round was repeated inside the third with the same seeds and gave the same S_{21} values to four decimals. Kaggle accepted every file and returned the scores of Table I, read back with the Kaggle CLI.

V. DISCUSSION

A. Which strategy should the platform select?

On the hold-out the hybrid has the best MAP@5 (0.3595) and the best public score (0.45244). It beats habit × schedule by 0.0229 [0.0197, 0.0261] and PPR by 0.0411 [0.0378, 0.0445] (Table I). Most of the gain comes from a better order of programmes the user already watches. The hybrid has the highest precision on familiar programmes (Table III), 83% of its slots are programmes the user watched before and only 5% are new to the user, while 35% of the actual targets are new. Habit × schedule needs no training and scores 0.42188 on the public board. Light users are hard for every method: on S_{21} the hybrid scores 0.2399 on the lightest third of users and 0.4815 on the heaviest.

Table IV

TOP 5 OF EVERY METHOD ON S_{21} FOR USERS A (3525695269081370558, 350 LABELLED HOURS AND 47 FULLY WATCHED PROGRAMMES IN WEEKS 11 TO 20) AND B (14049050536217794294, 59 HOURS, 14 PROGRAMMES), CHOSEN BEFORE ANY LIST WAS SEEN AS THE TEST USERS CLOSEST TO THE 90TH PERCENTILE OF LABELLED HOURS AND TO THE 10TH PERCENTILE AMONG THOSE WITH AT LEAST ONE FULLY WATCHED AIRING. BOLD: IN THE ACTUAL TOP FIVE OF WEEKS 21 TO 30. NO MARK: WATCHED FOR 0.8 BEFORE; t: ONLY TOUCHED; n: NEVER TOUCHED. LAST COLUMN OF EACH BLOCK: AP@5. FOR A EVERY METHOD FILLED ALL FIVE PLACES ITSELF; FOR B EVERY METHOD FILLED ALL FIVE PLACES ITSELF. TITLES AND CATEGORIES FROM THE SCHEDULE ARE LISTED BELOW THE TABLE; THE TEST-WINDOW LISTS ARE IN `MODELLING/ANALYSIS.IPYNB`.

	User A						User B					
Method	1	2	3	4	5	AP	1	2	3	4	5	AP
Actual top 5	**12001682**	**12002856**	**12001734**	**2400508**	**2400467**		**90082736**	**90072084**	**90057266**	**1000381**	**90055475**	
Popularity	2400480	240081	**2400467**	**2400508**	**12001682**	0.29	2400480^{t}	240081	2400467	2400508^{t}	12001682^{t}	0.00
Habit × sched.	**12001682**	**12002856**	**12001734**	2400480	**2400508**	0.76	2500413	2400467	240081	10003012	1100411	0.00
ALS	2400480	240081	**12001682**	**2400467**	20088	0.17	240081	2400467	2500413	400361	2400480^{t}	0.00
PPR	**12001682**	2400480	**12001734**	**2400508**	**2400467**	0.64	2500413	2400467	400361	240081	400335	0.00
Assoc. rules	2400480	240081	**2400467**	**2400508**	**12001682**	0.29	240081	2400480^{t}	2400467	2400508^{t}	20088^{n}	0.00
Content	5900319^{n}	12002720^{n}	12002904^{n}	120079707^{n}	120082346^{n}	0.00	5900319^{n}	5900347^{n}	2500413	2400480^{t}	2400467	0.00
Hybrid	**12001682**	**12002856**	2400480	**2400508**	**2400467**	0.71	2400467	2400480^{t}	240081	500331^{t}	10002893	0.00

20088 'Завтрак с 1+1'. Информационно-развлекательная программа (entertainment); 240081 Факты. День (news); 400335 'Хата на тата', 7 сезон, 1 эп (entertainment); 400361 'МастерШеф', 7 сезон, 17 эп (factual); 500331 'Один за всех' (entertainment); 1000381 Т/с 'CSI: Место преступления', 7 сезон, 6 и 7 с (series); 1100411 Премьера. 'Вещдок. Особый случай. Око за око' (factual); 2400467 'Викна-Новости' (news); 2400480 'ТСН. Неделя' (news); 2400508 Сегодня. Итоги с Олегом Панютой (news); 2500413 Гражданская оборона (news); 5900319 Абзац! (news); 5900347 'Деньги' (news); 10002893 Т/с 'Когда мы дома' (series); 10003012 Т/с 'Однажды в Одессе' (series); 12001682 М/с 'Губка Боб Квадратные Штаны' (kids); 12001734 М/с 'Волшебные покровители', 154 с. 'Хочу это вырезать / Мэрский день' (kids); 12002720 М/с 'Соник Бум' (kids); 12002856 М/с 'Элвин и бурундуки' (kids); 12002904 М/с 'Рыцари Нексо Найтс' (kids); 90055475 Х/ф 'Парк Юрского периода 3' (film); 90057266 Х/ф 'Терминатор 3: Восстание машин' (film); 90072084 Х/ф 'РЭД' (film); 90082736 Х/ф 'Человек-паук. Вдали от дома' (film); 120079707 М/ф 'Король обезьян' (kids); 120082346 М/ф 'Муми-тролли и зимняя сказка' (kids).

We read these numbers as follows; none of the explanations was tested by a decomposition. The target counts repeated full viewings, so it is dominated by programmes that air often and that the user already finishes. *TSN* is in the target five of half of the hold-out users, and news make 28% of target pairs against 2.6% of labelled airtime (EDA). This is why popularity alone reaches 0.2341, and why association rules mostly find the same programmes again (on average 3.85 of their 5 are in the popularity five, and 70% are news): a rule from one news programme to another has high confidence for almost every user. ALS pulls the user towards what similar users finish. With the strong regularisation selected on the hold-out it recommends only 78 distinct programmes and loses the user's smaller habits, which is where it falls behind PPR. PPR keeps half of the walk's mass on the user's own history, so 96% of its slots are familiar. The diffusion then orders those programmes by co-viewing centrality, while habit × schedule orders them by how often the user finishes them; this costs PPR against habit × schedule but gives it the widest catalogue coverage (730 programmes). Content attributes describe films well, but the target is mostly news and cartoons. Among new target programmes the user's favourite category matches 23.1% of the time, against 21.6% expected from category popularity alone (EDA), so content-based comes last. The schedule matters because a programme that stops airing cannot be watched: in the EDA, no top-five programme that stopped airing stayed in the next target.

We propose to serve the hybrid as the personalised "your programmes" row and to keep habit × schedule as the fallback when the model is unavailable. The current lists should not be presented as discovery. An unfamiliar programme in them is mostly a wrong guess (precision on new programmes is at most 0.157), and nothing here shows whether a viewer would enjoy it. A separate row with premieres in the viewer's usual channels and hours could be tested online; we have not measured it. The cost is low at this scale: the final notebook builds candidates and features for six snapshots and trains all models in 1,488 s on a laptop.

B. Failure case

The first table of list properties reported for habit × schedule "share watched at 0.8 before 0.0025, share new to the user 0.9975". A method that recommends almost only watched programmes cannot have these values, and user A's *SpongeBob*, watched 32 times, was marked "new". The cause was in how the sets of (user, programme) pairs were built: `DataFrame.to_numpy()` over a `uint64` and an `int64` column returns `float64`, because NumPy promotes the two types to a common one. Twenty-digit user ids exceed 2^{53}, so they were rounded and never matched. We now build the pairs from `zip(col.tolist())`. After the fix the share is 0.958, A's programmes are marked watched, and MAP@5, which is computed on another path, is unchanged. An earlier failure affected only the run time. The first ALS grid used over two hours of CPU without finishing, because a multi-threaded BLAS spent its time switching threads on one 64×64 solve per user. With BLAS limited to one thread a fit takes 4 to 47 s.

C. What worked, what surprised, limitations

Three things worked: rebuilding the organisers' target exactly, scoring by the expected airings in the coming schedule, and the habit-slot features, which carry most of the ranker's gain. We did not expect content-based to fall so far below popularity or more candidates to bring no gain, and we did not expect the public score to be 0.08 to 0.09 above the hold-out for every method except content-based. The main

limitation is that only 10.8% of schedule rows carry an id, so news and shows aired under unlabelled titles can be neither recommended nor counted. The hold-out target comes from a log with a gap that affects 23.9% of users. 67.9% of hold-out users have a tie between places five and six, and the organisers do not say how ties are broken, so part of the target is noise to every method. A programme missing from this log may still have been seen elsewhere. All results are offline and come from one platform and one season.

D. Next improvement

Instead of the linear coverage sums used now, predict for each user and airing the probability of covering 0.8 of it from the user's channel-hour habits, and sum these probabilities over the window's airings. The coverage features are already the strongest predictors, and their linear form ignores that a two-hour film needs almost all of its hours covered.

VI. AI Usage Disclosure

A. AI tools

Claude Opus 5.5 (`claude-opus-5-5`) in Claude Code 2.1.304, VS Code extension.

B. How AI was used

Code: all of `src/`, the notebooks, `tools/`, `tests/` and the report scripts. *Analysis and runs:* EDA, every experiment, Kaggle submissions with the CLI. *Debugging:* the failures of Section V-B and the Windows process issues. *Search and reading:* the competition data page, the course guidelines and the cited papers. *Drafting:* the whole text of this report.

C. What I personally verified

I hand-computed AP@5 of user A for the hybrid from Table IV: the hits are at ranks 1, 2, 4 and 5, so AP@5 $= (1/1 + 2/2 + 3/4 + 4/5)/5 = 0.71$, the value in the table. On the Kaggle submissions page `ranker_v3.csv` shows a private score of 0.44757 and a public score of 0.45244, as in Table I; Fig. 2 is my screenshot of that page. I ran `python tests/check_core.py`, and every check printed "ok", including the format check of all 12 submission files. I checked that `modelling/baselines.ipynb` reports MAP@5 0.2341 for popularity and 0.3366 for habit $\times$ schedule on S_{21}, as in Table I. I opened the DOIs of [1], [2], [3], [5], [7] and the links of [4], [6], [8]; authors, venues and years match the reference list.

D. What I trusted without verification

The join was checked on a toy schedule and by the clock-alignment test, not line by line on real data. I checked the bibliographic records of all references but did not read the papers myself; in the session the AI read the text of [1], [7], [5] and only the records or publisher pages of the others. We read the data page's "screen time" as a share per airing; a share per programme would change the target. The test window and its target are hidden, so all statements about weeks 31 to 42 rest on the returned scores.

E. Session log reference

The attached session export is the unmodified log of the Claude Code session in which this work was done.

References

[1] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22.

[2] G. Jeh and J. Widom, "Scaling personalized web search," in *Proc. 12th Int. Conf. World Wide Web (WWW)*, 2003, doi: 10.1145/775152.775191.

[3] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1993, pp. 207–216, doi: 10.1145/170035.170072.

[4] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[5] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. 10th ACM Conf. Recommender Systems (RecSys)*, 2016, pp. 191–198, doi: 10.1145/2959100.2959190.

[6] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," Microsoft Research, Tech. Rep. MSR-TR-2010-82, 2010. [Online]. Available: https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.

[8] sweet.tv, "sweet.tv — TV program recommender," Kaggle community competition, 2020. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender. [Accessed: 2026-10-21].